Assignment 2: Maze Solver with BFS, DFS, and A*

Daniel Graham (620171166)

The University of the West Indies, Mona

COMP3220 – Principles of Artificial Intelligence

Mr. Matthew Stone

November 23, 2026

Maze and Design Choices

I used the benchmark maze only, parsed from strings into a 2D list (1 = wall, 0 = open). All search code logic is in maze_solver.py and returns (path, visited_order, stats). visualize.py only draws what those functions return.

For the colours: navy blue walls, light gray open cells, green start node, red goal node, light-orange visited cells and purple final path.

The Greedy Best-First search algorithm was added to maze_solver.py to satisfy the Creative Enhancement requirement.

Comparison Table

Algorithm	Path Found?	Path Length	Nodes Expanded	Path Cost (A* only)
BFS	Yes	15	43	
DFS	Yes	23	31	
A*	Yes	15	33	14
Greedy Best-First	Yes	15	15	

Reflection

I. Of the three given original algorithms, the one that performed best on the benchmark maze is the A* search with the fewest expansions among the algorithms with 33 and a path length of 15. However, once added, the Greedy Best-First algorithm performed the best with 15 expanded nodes and a path length of 15.

II. DFS and BFS did not find the same path. DFS commits to one direction until blocked, so it takes a more winding route of 23 cells. BFS expands by distance so the first time it reaches the goal node is via a shorter route of 15 nodes.

III. The most interesting part for me was learning how to represent the different algorithms in Python code.

Comprehension

1. BFS guarantees the shortest path in terms of number of steps. Does DFS make this same guarantee? Explain why or why not, referencing your results from Part B if helpful.

   Ans: No. DFS follows one branch as deep as possible and returns the first path it hits, not the shortest. On the benchmark it found a 23 move path versus BFS's 15.

2. Why is A* generally able to expand fewer nodes than BFS while still finding an optimal path? What property must the heuristic have for this to hold?

   Ans: A* orders nodes by f = g + h, so it prefers nodes that look closer to the goal and skips many that BFS would explore. It stays optimal if h is admissible, meaning it never overestimates the true remaining cost.

3. Describe a maze layout or scenario (not necessarily the benchmark maze) where DFS would perform very poorly compared to BFS and A*.

   Ans: Depth-First Search (DFS) would perform poorly on mazes that feature long, winding, dead-end-heavy paths or when the goal lies very close to the starting point down a different branch.

4. Suppose you added diagonal movement to the maze. What would need to change in your BFS/DFS implementations, and would the Manhattan-distance heuristic still be admissible for A*? Why or why not?

   Ans: Neighbour generation must also include add the 4 diagonal moves • expanding the maximum number of neighbours to 8.

   Manhattan distance would no longer be admissible for A* as it assumes you can only move horizontally and vertically. When diagonal movement is allowed, the true shortest distance between two diagonally adjacent cells is either root(2) ≈ 1.41 or 1.

   If the goal is 1 step up and 1 step right, Manhattan distance calculates the cost as 1 + 1 = 2. However, with diagonal movement, the goal can be reached in a single diagonal step with a cost of 1 or 1.41. Because 2 > 1.41 and 2 > 1, the Manhattan distance overestimates the actual cost, violating the condition for admissibility.